

Devoir noté

Structures de données en Java

HAI822I - 2026

Master Informatique, parcours Intégration de COonnaissances
Université de Montpellier

Présentation

Le projet porte sur l'étude expérimentale du choix d'une structure de données dans un programme Java. Dans un cas d'étude concret, on cherchera à répondre à la question suivante : pour quels usages et pour quelles répartitions d'opérations une structure est-elle un meilleur choix qu'une autre ?

Chaque groupe travaillera sur un **conteneur principal** stockant des objets métier (produits, livres, patients, événements, etc.). Ce conteneur devra prendre en charge plusieurs opérations usuelles, chacune étant effectuée avec une certaine fréquence. Deux structures de données, de nature différente, seront imposées pour implémenter ce conteneur. Le travail consistera à comparer ces deux solutions du point de vue du **temps d'exécution**, de l'**occupation mémoire** et de l'**adéquation aux usages**.

Travail demandé

Pour le cas d'étude choisi, il faut :

1. définir au choix des interfaces / classes / classes abstraites pour représenter le **conteneur** et les éléments stockés ;
2. générer des données réalistes et en quantité suffisante pour rendre les mesures significatives ;
3. implémenter le conteneur avec les deux structures imposées ; lors de cette implémentation, prévoyez de traiter les cas d'erreurs avec des exceptions, et lorsqu'une méthode s'y prête, imaginez un itérateur ;
4. mesurer, pour chaque structure, le coût d'un nombre significatif d'opérations de chaque type ;
5. mesurer l'occupation mémoire ;
6. exécuter plusieurs scénarios mixtes dans lesquels les pourcentages des opérations varient ;
7. rechercher des scénarios où chaque structure est tour à tour la meilleure ;
8. discuter les résultats en les reliant aux complexités théoriques.

Organisation

Le projet est à réaliser en **groupe de 1 ou 2 étudiants**.

Dans le cas d'un binôme, le document final doit comporter un court paragraphe indiquant la répartition du travail.

Usage d'un LLM

L'usage d'un LLM est **autorisé pour toutes les tâches du projet**, y compris :

- recherche d'idées ;
- conception de classes ;
- génération ou amélioration de code ;
- aide au débogage ;
- proposition de protocoles expérimentaux ;
- aide à l'analyse ;
- rédaction du document.

Cette autorisation s'accompagne de deux exigences :

- Traçabilité.** Indiquez quel(s) modèle(s) vous utilisez et à quelles dates. Vous devez conserver les prompts importants utilisés au cours du projet, ainsi que, si nécessaire, les réponses obtenues (avec la date et le modèle). Une sélection représentative devra être fournie en annexe ou dans un fichier séparé.
- Empreinte carbone.** Vous tâcherez également d'évaluer l'empreinte carbone de votre usage des LLM lors de ce TP (choisissez des sources récentes sur lesquelles vous appuyer et indiquez-les).
- Compréhension.** À chaque étape, vous devez vous assurer de comprendre ce qui a été proposé ou produit. Lors de l'examen terminal du module, des questions pourront vous être posées sur l'architecture du programme, l'intérêt des structures choisies, le fonctionnement du code, le protocole de mesure, l'interprétation des résultats.

Attendus méthodologiques

Les deux implémentations doivent offrir le même comportement fonctionnel. Les comparaisons devront être faites sur des jeux de données identiques et sur plusieurs tailles de test. Les scénarios mixtes devront être explicités par une répartition des opérations, par exemple :

$a\%$ d'ajouts, $b\%$ de retraits, $c\%$ de recherches, $d\%$ de parcours, $e\%$ de statistiques

avec

$$a + b + c + d + e = 100.$$

On attend au minimum :

- des tailles de données variées ;
- des mesures répétées ;
- des résultats présentés clairement ;
- une discussion des limites de l'expérience.

Délivrables

1. Code Java

Le rendu doit contenir tout le code source nécessaire à l'exécution :

- classes métier ;

- interface ou abstraction commune du conteneur ;
- deux implémentations fondées sur les structures demandées ;
- générateur de données ;
- outils de benchmark ;
- point d'entrée permettant de relancer les expériences.

Le code doit être compilable, structuré et lisible.

2. Document de résultats

Le document doit présenter de manière synthétique :

- le sujet choisi ;
- les opérations considérées ;
- les structures comparées ;
- les hypothèses de génération de données ;
- le protocole expérimental ;
- les tailles testées ;
- les scénarios de pourcentages ;
- les résultats obtenus ;
- leur interprétation ;
- le lien avec les complexités théoriques ;
- une conclusion sur les contextes où chaque structure est préférable ;
- une courte note sur l'usage des LLM (modèles, dates, etc.) ;
- une évaluation de l'empreinte carbone (avec le protocole de calcul expliqué)
- les prompts et conversations principales (avec modèles et dates), en annexe ou dans un fichier associé.

3. Vidéo

Une vidéo de **5 minutes maximum** doit montrer et commenter l'exécution du projet. Elle doit faire apparaître au moins :

- le problème traité ;
- le lancement des expériences ;
- quelques résultats significatifs ;
- une conclusion sur le choix des structures.

Cas d'étude proposés

Un sujet sera attribué à votre groupe. Pour celui-ci vous seront imposées : la classe conteneur ; les opérations dominantes ; les deux structures à comparer.

Attention aux spécificités de certaines structures qui vous demanderont de définir explicitement un ordre sur les éléments. Les énoncés suggèrent des attributs à prévoir dans la classe représentant les objets stockés. Les cas d'étude ne spécifient pas tout, complétez et écrivez dans votre rapport ce que vous avez ajouté comme hypothèses de travail et choix d'usage des structures.

#	Conteneur	Opérations typiques	Structures proposées
1	CatalogueProduits	ajout, retrait par identifiant, recherche par identifiant, listing complet, comptage par catégorie	ArrayList<Produit> vs HashMap<Integer, Produit>
2	Bibliotheque	ajout, suppression par ISBN, recherche par ISBN, listing complet, comptage par genre	LinkedList<Livre> vs TreeMap<String, Livre>
3	AgendaEvenements	ajout, annulation, recherche par identifiant, obtention de tous les événements par ordre chronologique, comptage par type	PriorityQueue<Evenement> vs HashMap<Integer, Evenement>
4	AnnuaireEtudiants	inscription, désinscription, recherche par numéro, listing complet, comptage par groupe	ArrayDeque<Etudiant> vs HashMap<Integer, Etudiant>
5	InventaireEntrepot	ajout, retrait par référence, consultation par référence, inventaire complet, comptage par zone	LinkedList<ArticleStock> vs HashMap<String, ArticleStock>
6	Mediatheque	ajout, suppression, recherche par identifiant, parcours sans doublons, comptage par genre	HashSet<Morceau> vs TreeMap<Integer, Morceau>
7	GestionTickets	création, fermeture, recherche par numéro, listing complet, comptage par statut	ArrayList<Ticket> vs TreeMap<Integer, Ticket>
8	FileUrgencesPatients	ajout, prise en charge du plus prioritaire, recherche d'un patient, obtention de tous les patients par ordre de priorité, comptage par service	PriorityQueue<Patient> vs HashMap<Integer, Patient>
9	RepertoireContacts	ajout, suppression, recherche par identifiant, listing complet, comptage par catégorie	ArrayList<Contact> vs HashMap<Integer, Contact>
10	SuiviColis	enregistrement, retrait, recherche par numéro, listing complet, comptage par statut	ArrayDeque<Colis> vs HashMap<String, Colis>
11	PlanificateurTaches	ajout, extraction de la tâche la plus urgente, recherche par identifiant, obtention de toutes les tâches par ordre de priorité, comptage par état	PriorityQueue<Tache> vs HashMap<Integer, Tache>
12	HistoriqueTransactions	ajout, annulation, recherche par identifiant, parcours chronologique, comptage par type	TreeSet<Transaction> vs TreeMap<Integer, Transaction>
13	ParcCapteurs	ajout, retrait, recherche par identifiant, inventaire complet, comptage par type	LinkedList<Capteur> vs HashMap<Integer, Capteur>
14	CollectionFilms	ajout, suppression, recherche par identifiant, listing trié, comptage par genre	TreeSet<Film> vs TreeMap<Integer, Film>
15	GestionReservations	ajout, annulation, recherche par identifiant, obtention de toutes les réservations par ordre de date ou de priorité, comptage par salle ou statut	PriorityQueue<Reservation> vs HashMap<Integer, Reservation>
16	RegistrePatients	ajout, sortie d'un patient, recherche par identifiant, listing complet, comptage par service	LinkedList<Patient> vs HashMap<Integer, Patient>
17	GestionExamens	ajout d'un examen, suppression, recherche par code, parcours trié par date, comptage par matière	TreeSet<Examen> vs HashMap<String, Examen>
18	FileImpression	ajout d'un travail, impression du plus prioritaire, recherche par identifiant, obtention de tous les travaux par ordre de priorité, comptage par type	PriorityQueue<TravailImpression> vs HashMap<Integer, TravailImpression>
19	DepotDocuments	ajout d'un document, retrait, recherche par référence, listing complet, comptage par thème	ArrayList<Document> vs TreeMap<String, Document>
20	GestionSalles	ajout d'une salle, retrait, recherche par numéro, inventaire trié, comptage par bâtiment	TreeSet<Salle> vs HashMap<Integer, Salle>